Six Layers of Self-Improvement: A Framework for Decomposing and Governing Recursive AI Enhancement

Farooq Abdul Rahim, Nithish Mohan, Ajomon Jose

Draft v1.2.3 — March 2026

Abstract

Recursive self-improvement (RSI) is often treated as a single capability, which obscures mechanisms and blocks calibrated governance. We present a six-layer framework that decomposes RSI by modification substrate: self-evaluation (L1), strategy evolution (L2), tool optimization (L3), knowledge synthesis (L4), architecture adaptation (L5), and meta-learning (L6). We model two coupled feedback loops: an inner loop (L4↔L2) that converges to a logistic ceiling under perturbation, and an outer loop (L6↔L1) whose behavior depends on bounded meta-learning yield and architecture ceiling-lift parameters. The analysis yields a four-regime phase diagram spanning bounded to transiently superlinear improvement. We extend the model to multi-dimensional quality, extraction efficiency decay, non-stationary environments, concurrent execution, and cycle economics. We derive seven falsifiable predictions with sensitivity analysis and propose a depth-calibrated governance architecture with cumulative drift detection, governance-gate error analysis, and alignment to the "three lines of defense" model and the EU AI Act. This is a theoretical contribution; no empirical experiments have yet been run. To enable empirical testing, we provide a ten-experiment validation protocol with concrete systems, metrics, and decision criteria.

Keywords: recursive self-improvement, AI safety, AI governance, meta-learning, intelligence explosion, layered architecture

1. Introduction

Artificial systems that improve their own capabilities have been a central AI-safety concern since Good's (1965) "ultraintelligent machine" argument. The core scenario is recursive: each improvement cycle raises capability and accelerates subsequent cycles, potentially producing an intelligence explosion that outpaces human oversight (Bostrom, 2014; Chalmers, 2010).

Despite decades of debate, RSI is still often modeled as an undifferentiated capability. This binary framing obscures that shallow self-improvement already occurs in deployed systems and collapses qualitatively different activities (e.g., tool selection vs. architecture change) into one category, preventing calibrated governance.

We model self-improvement as a *spectrum of depth*: six layers that operate on different substrates, follow different dynamics, and require different governance controls.

Distinguishing from standard change management. Tiered approvals are not new (e.g., ITIL), but RSI introduces compounding dynamics that ordinary change control does not formalize.

Our contribution adds three RSI-specific elements: (1) explicit feedback-loop analysis of cross-layer compounding (Eq. 3), (2) a phase diagram identifying when improvement velocity becomes dangerous (Proposition 5), and (3) compositional-risk detection for salami-sliced modification sequences (Section 5.7).

The framework makes five contributions:

1. A **taxonomy** that decomposes RSI into six substrate-specific layers using explicit substrate-independence and governance-discontinuity criteria (Section 3).
2. A **formal model** of inner and outer feedback loops, with analytical convergence results and a two-parameter phase diagram of improvement regimes, extended to multi-dimensional quality, extraction efficiency, non-stationarity, concurrency, and cycle economics (Section 4).
3. A **governance architecture** that scales oversight by depth, including compositional-risk detection, governance-gate error analysis, governance horizons, and constitutional-rule interaction (Section 5).
4. Seven **falsifiable predictions** with sensitivity analysis for both dynamics and governance efficacy (Section 4.10).
5. A **ten-experiment validation protocol** with concrete systems, metrics, success criteria, and timelines to test the framework empirically (Section 7).

Scope and limitations. This manuscript presents a theoretical contribution. No empirical experiments have been conducted, no systems have been built, and no new data has been collected. Propositions 1–10 are mathematical results under stated assumptions. Section 7 specifies how each claim can be tested in practice; empirical results will be reported in subsequent versions.

---

2. Related Work

2.1 Recursive Self-Improvement

Foundational work spans Good's (1965) "ultraintelligent machine," Schmidhuber's (2003) Gödel Machine, Bostrom's (2014) treatment of intelligence-explosion consequences, Yampolskiy's (2015) complexity-limit argument, and Hutter's (2012) analysis of computational constraints. Collectively, these works motivate RSI risk but leave the internal improvement structure largely unspecified.

Recent systems make RSI operational: Gödel Agent (Yin et al., 2024), Darwin Gödel Machine (Hu et al., 2025), LADDER (Simonds et al., 2025), SELF (Lu et al., 2023), MARS (Wang et al., 2026a), and RISE (Qu et al., 2024). They demonstrate practical self-improvement mechanisms, but typically treat RSI as a single class of capability.

Retrospective classification of existing systems. To illustrate immediate utility, we classify four representative systems:

- **Reflexion** (Shinn et al., 2023) sits at L1–L2 via verbal self-evaluation and strategy refinement; its L1 diagnostics are informal relative to our structured taxonomy.
- **Voyager** (Wang et al., 2023) spans L2–L3 through strategy development and an expanding skill/tool library.
- **AlphaEvolve** (DeepMind, 2025) is a boundary case: L5 when seen as internal architecture improvement, or L3 when evolved algorithms remain external tools.
- **Darwin Gödel Machine** (Hu et al., 2025) spans L2–L5 with nascent L6 through code-level modification and evolutionary search over improvement operators.

No existing deployed system operates at sustained L6 depth with the outer feedback loop active.

2.2 Existing Decompositions and Taxonomies

Several works decompose AI systems or self-improvement into categories. Table 1 distinguishes our contribution.

Table 1: Comparison with existing decompositions

Framework	What is decomposed	Categories	Relation to our work
Wikipedia RSI taxonomy	Improvement *capability level*	Basic / Weak / Strong	Classifies *how much*; we classify *what substrate*
7 Layers of Agentic AI (2025)	Agent *architecture*	Memory → orchestration	Decomposes what the system *is*; we decompose how it *changes itself*
Agentic AI taxonomy (Gangadharan et al., 2026)	Agent *components*	Perception, brain, planning, action, tool use, collaboration	Architecture-focused; no governance or feedback analysis
ICLR 2026 RSI Workshop	RSI *concerns*	Operators, diagnostics, governed adaptation	Three high-level concerns; we operationalize into six layers
Five-layer governance (Agarwal & Nene, 2025)	*Governance* structure	Regulation → audit	Governs AI deployment; we govern AI *self-modification*
This paper	Self-improvement *depth*	L1–L6: substrate-specific	Decomposes by substrate, maps to governance, models dynamics

The critical distinction: existing taxonomies decompose agent *architecture* or *governance structure*, while ours decomposes the *self-improvement process*. These are orthogonal in the sense that any

architecture *possessing the relevant substrates* can engage in self-improvement at the corresponding depths. A system without an explicit knowledge store cannot operate at L4; a system without modifiable architecture cannot operate at L5. The layers thus serve both as a taxonomy of self-improvement types and implicitly as a specification of the architectural prerequisites for each depth.

2.3 Meta-Learning

Meta-learning provides the computational foundation for L6. The concept traces to Schmidhuber (1987) and has been extensively developed (Finn et al., 2017; Hospedales et al., 2022). Our framework draws a distinction this literature typically does not: between meta-learning as *one component* of a self-improving system (L6) and the five object-level layers it operates *upon*.

2.4 AI Governance Frameworks

Existing frameworks — the EU AI Act (European Parliament, 2024), the OECD AI Principles (OECD, 2024), and multi-layered proposals (Agarwal & Nene, 2025; Joshi, 2025) — govern AI *outputs* and *deployments* but not the *self-modification process*. Schuett (2023) adapted financial regulation's "three lines of defense" to AI risks. Our framework extends this to self-improvement governance specifically (Section 5.5).

2.5 Multi-Objective Optimization

The formal model's extension to multi-dimensional quality (Section 4.0) connects to the Pareto optimization literature. In multi-objective optimization, convergence means approaching a Pareto frontier rather than a scalar optimum (Deb, 2001; Miettinen, 1999). Recent work in multi-objective reinforcement learning (Hayes et al., 2022) and Pareto-optimal model selection (Karl et al., 2023) addresses the trade-off navigation problem in ML-specific contexts. Our contribution is applying this formalism to self-improvement dynamics: the inner loop converges to a Pareto surface bounded by the architecture ceiling, and the outer loop migrates the surface outward.

---

3. The Six-Layer Taxonomy

3.1 Overview

We model self-improvement as a depth-ordered stack of six layers in three tiers (Figure 1):

[**FIGURE 1: The six-layer stack with two feedback loops, governance boundary, and improvement constitution.**]

Tier	Layers	Substrate modified	Risk level
Surface optimization	L1, L2, L3	Evaluation criteria, decision heuristics, external tools	Low to moderate
Deep restructuring	L4, L5	Knowledge structures, computational architecture	Significant to high
Recursive transcendence	L6	The improvement process itself	Maximum

3.2 Derivation Principles: Why Six Layers

The decomposition is derived by combining two criteria.

Criterion 1: Substrate independence. Activities belong to different layers when they modify different substrates that can be changed independently. We identify six substrates: (i) performance model, (ii) decision procedures, (iii) external tool interface, (iv) accumulated knowledge, (v) computational structure, and (vi) improvement methodology.

Criterion 2: Governance discontinuity. At each substrate boundary, the governance model changes materially: auditing (L1), inspectability (L2), capability-expansion gating (L3), content audit (L4), sandboxed testing (L5), and human-in-the-loop control (L6). A coarser decomposition loses operationally relevant governance distinctions; a finer decomposition adds distinctions without governance value.

Completeness. Any self-modification should fall into one of these six categories; we invite counterexamples that identify a distinct, governance-relevant seventh substrate.

On logical versus causal independence. The criterion requires *logical* independence (substrates can be modified separately), not *causal* independence (no downstream effects). Causal coupling is expected: L4 affects L2, L3 affects L4, and L5 can invalidate prior L4 knowledge. The framework manages these couplings through cross-layer invalidation (Section 4.5), escalation signaling (Section 4.5.2), and cumulative drift detection (Section 5.7).

Depth ordering. We define depth structurally:

> **Definition 1 (Depth ordering).** Layer L_i is deeper than L_j (written $L_i \succ L_j$) if substrate s_i constrains the maximum achievable quality at substrate s_j, but not vice versa.

Under this definition, s_5 constrains what can be achieved at s_4, and s_6 constrains improvement rates across lower layers; thus L5 $\succ$ L4, L4 $\succ$ L2, and L6 $\succ$ L5. L2 and L3 are comparable (neither structurally dominates the other) but remain separate because they target different substrates with different governance requirements.

This makes depth a property of architecture-level constraint relationships, not of historical improvement outcomes.

3.3 Layer Definitions

Layer 1: Self-Evaluation & Diagnostics Substrate s_1: The system's model of its own performance.

L1 evaluates outcomes after every task, classifying failures (knowledge gap, strategy failure, tool failure, resource constraint, specification ambiguity, coordination failure), performing bottleneck identification, and tracking confidence calibration.

Risk: Low in isolation. Becomes significant when self-evaluation becomes *self-modeling* — understanding "what am I, and what constraints are placed on me" — which shades into situational awareness (Laine et al., 2024). **Governance boundary:** self-evaluation should measure output quality, not system architecture or oversight mechanisms.

Layer 2: Strategy Evolution Substrate s_2: The system's reasoning patterns and decision heuristics.

L2 maintains a strategy library and develops micro-heuristics through counterfactual reasoning over both successful and failed tasks.

Risk: Materializes when strategy evolution includes strategies for managing *oversight*. A system developing "when monitored, use approach A; when unmonitored, use approach B" has crossed into deceptive alignment (Hubinger et al., 2019). **Governance boundary:** strategy libraries must be inspectable; any strategy referencing oversight conditions must trigger an alert.

Layer 3: Tool & Integration Optimization Substrate s_3: The system's interface to external capabilities.

L3 optimizes tool selection, composition (pipelines, fan-out/fan-in, fallback chains, ensembles, adaptive routing), and in advanced cases, tool creation.

Risk: Capability expansion through tool acquisition — accessing financial APIs, provisioning tools, or communication channels expands the action space beyond intent (Kinniment et al., 2023). **Governance boundary:** gate on new *categories* of action, not performance within existing categories.

Layer 4: Knowledge Synthesis & Compression Substrate s_4: The system's distilled knowledge structures.

L4 performs pattern extraction, causal modeling, knowledge compression, and cross-domain transfer. This marks the transition to deep restructuring.

Risk: The system may synthesize knowledge about its *own constraints*, or develop *emergent knowl-*

edge no human anticipated. **Governance boundary:** periodic audit of synthesized principles; any principle referencing the system's own constraints must trigger review.

Layer 5: Architecture Adaptation **Substrate s_5:** The system's computational structure.

L5 modifies pipelines, model composition, memory architecture, and compute allocation. Changes expand the capacity ceiling K for all other layers.

Risk: Hardest to evaluate, predict, and reverse. **Governance boundary:** sandbox, test, human-approve all changes; maintain a canonical snapshot for rollback.

Layer 6: Meta-Learning **Substrate s_6:** The improvement process itself.

L6 modifies improvement allocation, diagnostic quality, learning rate, transfer capability, and experiment design. Its impact is compounding: improving the improvement rate amplifies all subsequent cycles.

Risk: Maximum. Failure modes: improvement velocity exceeding oversight velocity, goal drift through meta-optimization, opacity cascade. **Governance boundary:** human-in-the-loop always, hard rate limits, externally locked improvement criteria.

---

4. Dynamics: Formal Model, Feedback Loops, and Predictions

4.0 Formal Preliminaries

Definition 2 (Self-Improvement Engine). An SIE is a tuple (S, L, Φ, Γ, C) where $S = (s_1, \ldots, s_6)$ is the system state partitioned by substrate, $L = \{L_1, \ldots, L_6\}$ is the set of layer functions, Φ is the set of feedback functions, Γ is the governance gate, and C is the improvement constitution (read-only).

Definition 3 (Improvement cycle). A single cycle at layer i: (1) observe traces E and state S, (2) generate candidate Δ_i, (3) evaluate $\Gamma(\Delta_i, C)$; reject terminates, (4) experiment in sandbox, (5) apply $s_i \leftarrow s_i + \Delta_i$ if success criteria met; rollback otherwise.

Definition 4 (Active depth). $d(t) = max\{i : L_i$ produced a validated positive change in $[t - w, t]\}$.

Definition 5 (Improvement). A candidate Δ_i constitutes an improvement if the following conditions hold jointly:

(a) *Weighted net-impact criterion:* $net\_impact(\Delta_i) = \Sigma_j\ w_j \cdot score(\delta_j) \cdot penalty(regressions) > \theta$, where $\theta > 0$ is the practical significance threshold (the dead zone), w_j are the constitutionally-defined dimension weights, $score(\delta_j)$ normalizes the raw change against the baseline variance, and $penalty(regressions)$ is a multiplicative factor that decreases with each soft regression.

(b) *No hard-regression veto:* For all dimensions j, $\delta_j > -H\_j$, where H_j is the hard-regression threshold for dimension j. A candidate that causes any single dimension to degrade past its hard-regression threshold is rejected regardless of net impact.

(c) *Statistical confidence:* The observed net impact exceeds zero with confidence $\geq 95\%$ ($p < 0.05$ with multiple-testing correction across dimensions).

This definition anchors the dynamics model to the evaluation framework. When we write $q(t+1) > q(t)$ in the convergence proofs, we mean an improvement in the sense of Definition 5: a statistically significant, multi-dimensionally evaluated, net-positive change that passes the dead zone threshold and triggers no hard-regression veto. The convergence target is not the maximum of $q(t)$ but the maximum achievable through such improvements — a distinction that matters because the dead zone and hard-regression veto make some regions of quality space unreachable through incremental improvement.

Operationalizing the learning rate α. The parameter $\alpha \in (0, 1)$ appears in all dynamics equations and is labeled "learning rate," but its concrete meaning warrants specification. We define α as a composite:

$$\alpha = p\_gen \cdot p\_gov \cdot p\_exp$$

where p_gen is the probability that the layer engine generates a candidate that addresses the identified gap (candidate generation success rate), p_gov is the probability that the candidate passes the governance gate (governance pass-through rate), and p_exp is the probability that the candidate produces a validated improvement in experimentation (experiment success rate). This decomposition connects α to measurable operational quantities and makes Prediction 5 (meta-learning acceleration) more precise: L6 improving α means improving one or more of these three components — e.g., better candidate generation targeting, reduced governance false-positive rates, or better experiment design that detects real improvements more reliably.

Table 2: Notation summary

Symbol	Meaning	Domain	Introduced
$S = (s_1,...,s_6)$	System state, partitioned by substrate	—	Def. 2
L_i	Layer i function	$S \times E \to \Delta_i$	Def. 2
Φ	Feedback function set (inner, outer)	—	Def. 2
Γ	Governance gate	$\Delta_i \times C \to$ *{approve, reject, escalate}*	Def. 2
C	Improvement constitution (read-only)	—	Def. 2
Δ_i	Proposed change to substrate s_i	—	Def. 3
E	Set of execution traces	—	Def. 3
$d(t)$	Active improvement depth at time t	{1,...,6}	Def. 4
w	Observation window for depth calculation	Time	Def. 4
θ	Practical significance threshold (dead zone)	$\mathbb{R}^+$	Def. 5
$\mathbf{q}(t)$	System quality vector at step t	$\mathbb{R}\hat{}m$	§4.0.1
$q(t)$	Scalar aggregate quality at step t	[0, K_max]	§4.0.1
$\alpha(t)$	Learning rate at step t	(0, 1)	§4.0
p_gen, p_gov, p_exp	Components of α	(0, 1)	§4.0
$K(t)$	Architecture ceiling at step t	(0, K_max]	Eq. 1
K_max	Physical maximum quality	$\mathbb{R}^+$	§4.2
$\eta(t)$	Knowledge extraction efficiency at step t	(0, 1]	Eq. 1
$\epsilon(t)$	Environmental perturbation at step t	$\mathbb{R}\geq^0$	§4.0.2
β	Meta-improvement rate (L6 parameter)	$\mathbb{R}\geq^0$	§4.2
γ	Ceiling-lift rate (L5 parameter)	$\mathbb{R}\geq^0$	§4.2
$g(t)$	Meta-learning yield per cycle	$\mathbb{R}\geq^0$	§4.2
$h(t)$	Architecture improvement yield per cycle	$\mathbb{R}\geq^0$	§4.2
w_j	Weight for quality dimension j	(0, 1), $\Sigma w_j = 1$	§4.0.1
K_j	Architecture ceiling for dimension j	(0, K_max,j]	§4.0.1
$\rho(\Delta)$	Cost of improvement cycle producing Δ	$\mathbb{R}\geq^0$	§4.9

4.0.1 Multi-Dimensional Quality In practice, system quality is not a scalar. An improvement that increases accuracy may increase latency; a cost reduction may decrease reliability. We define quality as a vector:

$$\mathbf{q}(\mathrm{t}) = (\mathrm{q}_1(\mathrm{t}), ..., \mathrm{q}_m(\mathrm{t}))$$

where each $q_j(t)$ represents performance on a distinct dimension. We identify five operationally relevant dimensions: *quality* (output correctness), *speed* (task completion time), *cost efficiency* (per-task resource consumption), *reliability* (success rate and error rate), and *tail behavior* (worst-case performance at the 95th and 99th percentiles).

Each dimension has its own architecture ceiling K_j and a configurable weight w_j with $\Sigma w_j = 1$. The scalar aggregate used in the dynamics model is:

$$q(t) = \Sigma_j \; w_j \cdot q_j(t)$$

Proposition 1 (Per-dimension convergence). *Under fixed architecture (s_5 constant), constant η, fixed trade-off weights, and no environmental perturbation, each dimension $q_j(t)$ converges independently:*

$$q_j(t + 1) = q_j(t) + \alpha \cdot [K_j - q_j(t)] \cdot \eta_j(t)$$

where $\eta_j(t)$ is the extraction efficiency for dimension j. lim_{t→∞} $q_j(t) = K_j$ for all j, with exponentially decaying increments per dimension.

Proof. Identical to Proposition 3 below, applied per dimension.

Remark (Pareto convergence). When dimensions trade off against each other — that is, when improving q_j requires accepting a decrease in q_k — the per-dimension convergence statement is insufficient. The system instead converges to a *Pareto surface*: the set of quality vectors $\mathbf{q}$ such that no dimension can be improved without degrading another, given the current architecture. The inner loop's convergence under trade-offs is to this Pareto surface, not to the vector of dimension-specific ceilings (K_1, ..., K_m).

The practical consequence is that an SIE operating in a trade-off regime must make *allocation decisions* — which dimension to prioritize — in addition to improvement decisions. These allocation decisions are governed by the weight vector $\mathbf{w}$, which is stored in the improvement constitution and modifiable by operators but not by the SIE itself. Adaptation of $\mathbf{w}$ is itself an L6 activity, subject to meta-learning governance (see Section 5.4.1 for the interaction between $\mathbf{w}$ adaptation and the objective immutability rule). This prevents the system from drifting toward a quality profile that differs from operator intent.

For tractability in the dynamics model, we work with the scalar aggregate $q(t)$ in the remainder of this section. All propositions hold per-dimension under the per-dimension convergence statement above. Cross-dimensional trade-offs introduce complications beyond the scalar model's scope; we return to this in the economics of improvement (Section 4.9).

4.0.2 Quality Dynamics and Environmental Non-Stationarity We model the basic quality dynamics as:

$$q(t+1) = q(t) + \alpha \cdot [K - q(t)] \cdot \eta(t) \quad (1)$$

where $\alpha \in (0, 1)$ is the learning rate (decomposed in Section 4.0 above), K is the **architecture ceiling** (maximum quality under the current architecture s_5), and $\eta(t) \in (0, 1]$ is the **knowledge extraction efficiency** — the fraction of the remaining quality gap that the system can close per cycle. Section 4.1 analyzes this equation's convergence properties in detail.

In production, the task environment is not stationary: user behavior shifts, APIs deprecate, regulations update, competitors adapt. An environmental shift can *reset* the effective q(t): a policy change invalidates knowledge, a tool deprecation disables capabilities, and a traffic pattern shift renders strategies suboptimal. For example, a client policy update in a customer support system could cause quality to drop from 91% to 12% accuracy overnight — the system's knowledge becomes stale, but its confidence remains high.

We extend Equation 1 with an environmental perturbation term:

$$q(t+1) = q(t) + \alpha \cdot [K - q(t)] \cdot \eta(t) - \epsilon(t) \quad (1')$$

where $\epsilon(t) \geq 0$ is the environmental perturbation at step t, representing the quality loss from external changes the system has not yet adapted to.

Proposition 2 (Inner-loop convergence under perturbation). *Under fixed architecture (s_5 constant) and constant η in Equation 1', if $\epsilon(t) \to 0$ as $t \to \infty$ (perturbations are transient), then $q(t) \to K$. If $\epsilon(t) \to \epsilon > 0$* (persistent environmental change), the system converges to $q = K - \epsilon/(\alpha\eta)$, provided $\epsilon < \alpha\eta K$.

Proof. At equilibrium, q(t+1) = q(t), so $\alpha \cdot [K - q] \cdot \eta = \epsilon$. Solving: $q = K - \epsilon/(\alpha\eta)$. This equilibrium is stable because the improvement term $\alpha \cdot [K - q] \cdot \eta$ increases as q decreases below q *(stronger signal drives larger improvement) and* ϵ is constant. If $\epsilon \geq \alpha\eta K$, *then* $q \leq 0$ — the environment degrades faster than the system can improve, and the system cannot maintain positive quality.

Remark (Red Queen dynamics). When $\epsilon(t)$ is persistent, the system must improve continuously just to maintain current quality — a phenomenon we term *Red Queen dynamics* by analogy with evolutionary biology. In this regime, the system's observed behavior (continuous improvement cycles with stable quality) may mislead observers into thinking the system is improving when it is merely *keeping pace*. The distinction between net improvement (q(t) rising) and Red Queen maintenance (q(t) stable despite ongoing improvement cycles) is operationally important: a system in Red Queen mode should not trigger L4/L5 escalation for "failure to improve," because the improvement cycles are successfully preventing degradation.

Definition 6 (Red Queen detection). A system is in Red Queen mode at time t if: (a) improvement cycles are producing validated positive candidates ($\alpha > 0$), (b) environmental perturbation is ongoing ($\epsilon(t) > 0$), and (c) quality is approximately stable ($|q(t) - q(t-n)| < \delta$ for the last

n cycles). Red Queen mode is distinguished from η-stagnation (Definition 7) by the presence of ongoing perturbation: in stagnation, improvement stops because η decays; in Red Queen mode, improvement continues but is consumed by environmental adaptation.

4.1 The Inner Loop (L4↔L2): Bounded Improvement

The inner loop connects knowledge synthesis (L4) to strategy evolution (L2):

> Traces → L4 compresses into principles → Principles refine L2 strategies → Better outcomes → Richer traces

[FIGURE 2: Inner loop enclosed within bounded region set by ceiling K.]

In the absence of environmental perturbation ($\epsilon = 0$), each inner-loop cycle follows Equation 1:

$$q(t + 1) = q(t) + \alpha \cdot [K - q(t)] \cdot \eta(t)$$

If $\eta(t)$ is constant ($\eta(t) = \eta_0$), Equation 1 is a discrete logistic map with solution:

$$q(t) = K - (K - q_0) \cdot (1 - \alpha\eta_0)^t \quad (2)$$

Proposition 3. *Under fixed architecture (s_5 constant), constant η, and no environmental perturbation, the inner loop converges: lim_{t→∞} q(t) = K, with exponentially decaying increments.*

Proof. From Equation 2, $q(t+1) - q(t) = \alpha\eta_0(K - q_0)(1 - \alpha\eta_0)^t$. Since $(1 - \alpha\eta_0) \in (0, 1)$, this decays geometrically to zero.

The inner loop is where most "the system gets noticeably better over time" improvement occurs in production systems. It is powerful, safe, and bounded.

4.1.1 Extraction Efficiency Dynamics

Proposition 4 (Stagnation below ceiling). *If $\eta(t) = \eta_0 \cdot r^t$ for $r \in (0, 1)$, the system stagnates at*

$$q = \mathrm{K} - (\mathrm{K} - \mathrm{q}_0) \cdot \prod_{t=0}^{\infty} [1 - \alpha \cdot \eta_0 \cdot \mathrm{r}^t]^*$$

which satisfies $q < \mathrm{K}$ whenever $\mathrm{r} < 1$. The gap $\mathrm{K} - \mathrm{q}$ is positive and depends on the ratio $\alpha \cdot \eta_0$ / (1 − r): slower η decay (r close to 1) allows the system to approach K more closely before stagnating.

Proof sketch. The per-step increment is $\Delta(t) = \alpha \cdot \eta_0 \cdot r^t \cdot [K - q(t)]$. The cumulative improvement $\Sigma\ \Delta(t)$ converges (as a product of terms less than 1) to a finite value less than $K - q_0$ when $r < 1$. The system reaches q ***where the remaining gap is*** $K - q > 0$, but increments are effectively zero.

This proposition formalizes the practical observation that production AI systems experience diminishing returns that *feel* like a ceiling even though the theoretical architecture ceiling has not been reached. The distinction between stagnation (η-limited) and saturation (K-limited) is operationally important: stagnation is addressable by L4 improvements to pattern extraction or L6 meta-learning improvements, while saturation requires L5 architecture changes.

Definition 7 (Stagnation vs. saturation). A system is *η-stagnated* if $q(t) < K - \epsilon$ and the mean per-cycle increment over the last n cycles falls below δ_min (quality is below the ceiling but improvement has stopped). A system is *K-saturated* if $|q(t) - K| < \epsilon$ (quality has reached the ceiling). A system is in *Red Queen mode* if improvement cycles are active but quality is approximately stable due to ongoing environmental perturbation (Definition 6). The three conditions require different responses: η-stagnation → escalate to L4/L6; K-saturation → escalate to L5; Red Queen mode → maintain current improvement velocity.

4.1.2 Tool Contributions to the Inner Loop The inner loop is formally defined as L4↔L2, but L3 (tool optimization) contributes materially to quality improvements.

Remark 1 (L3 as an approximately additive term). L3 improvements contribute a quality increment $\tau(t)$ that is *approximately* independent of the L4↔L2 feedback cycle on the timescale of a single cycle:

$$q(t+1) = q(t) + \alpha \cdot [K - q(t)] \cdot \eta(t) + \tau(t) - \epsilon(t) \quad (4)$$

where $\tau(t) \geq 0$ is the L3 improvement increment at step t. On the timescale of a single cycle, tool improvements do not participate in the knowledge-strategy feedback loop: they resolve issues directly rather than through iterative refinement.

However, over multiple cycles, L3 improvements create second-order effects that feed the L4↔L2 loop. If an L3 tool fix resolves a class of failures, the system now *succeeds* on tasks it was previously failing, producing successful traces for cases L4 has never seen succeed before. These new traces contain extractable patterns. Similarly, if L3 adds a new tool capability, L2 now has a new action in its repertoire, changing which strategies are viable. The additive model (Equation 4) is therefore a first-order approximation: it is accurate per-cycle but underestimates L3's cumulative contribution over many cycles.

This first-order additive structure is why L3 is safe to omit from the inner-loop convergence proof (Proposition 3 holds with or without τ) while still being important in practice: L3 improvements often produce the largest *immediate* quality gains precisely because they bypass the iterative feedback cycle.

4.2 The Outer Loop (L6↔L1): Conditionally Bounded Acceleration

The outer loop links meta-learning (L6) with diagnostics (L1):

> L6 improves the improvement process → Better L1 diagnostics → Better opportunities → Better improvements → Better L6 updates

[FIGURE 3: Outer loop arc from L6 through L5 (ceiling-lifting) back to L1. Phase diagram inset showing Regimes A–D.]

Unlike the inner loop, the outer loop changes the inner-loop parameters:

- L5 raises the ceiling: $K(t + 1) = K(t) + \gamma \cdot h(t) \cdot [1 - K(t)/K\_max]$
- L6 raises adaptation speed: $\alpha(t + 1) = \alpha(t) + \beta \cdot g(t) \cdot [1 - \alpha(t)]$

with β the meta-improvement rate, γ the ceiling-lift rate, and $g(t)$, $h(t)$ per-cycle yields. Logistic terms keep $\alpha(t) < 1$ and $K(t) \leq K\_max$.

Substituting into Equation 1':

$$q(t + 1) = q(t) + \alpha(t) \cdot [K(t) - q(t)] \cdot \eta(t) - \epsilon(t) \quad (3)$$

Proposition 5 (Phase diagram). With $\epsilon = 0$ for regime characterization, Equation 3 has four regimes parameterized by (β, γ):

- **Regime A ($\beta \approx 0$, $\gamma \approx 0$):** Inner-loop behavior only; bounded convergence (or η-stagnation if η decays).
- **Regime B ($\beta > 0$, $\gamma \approx 0$):** Faster convergence via increasing α, still bounded by fixed K.
- **Regime C ($\beta \approx 0$, $\gamma > 0$):** Sustained growth via rising K, bounded by K_max.
- **Regime D ($\beta > 0$, $\gamma > 0$):** Simultaneous α and K growth can produce a transient super-linear phase before bounded convergence.

Proof. *Let* $\Delta q(t) = \alpha(t) \cdot [K(t)\text{-}q(t)] \cdot \eta(t)$ *and* $G(t) = K(t)\text{-}q(t)$ *(with ϵ=0). Then:*

$G(t+1) = G(t) \cdot (1\text{-}\alpha(t) \cdot \eta(t)) + \gamma \cdot h(t) \cdot (1\text{-}K(t)/K\_max)$.

In Regimes A/B (γ=0), $G(t)$ *contracts and quality converges (B faster than A). In Regime C, the ceiling-lift term sustains growth until* $K(t)$ *approaches* K_max. *In Regime D:*

$\Delta q(t+1)/\Delta q(t) = [\alpha(t+1)/\alpha(t)] \cdot [G(t+1)/G(t)] \cdot [\eta(t+1)/\eta(t)]$.

A superlinear transient requires $\Delta q(t+1)/\Delta q(t) > 1$, *which yields condition:*

$$\beta \cdot g(t) \cdot (1-\alpha(t))/\alpha(t) + \gamma \cdot h(t) \cdot (1-K(t)/K\_max)/G(t) > \alpha(t) \cdot \eta(t) \quad (\star)$$

As $\alpha \to 1$, $K \to K\_max$, *and yields decay, ($\star$) fails and dynamics revert to bounded convergence.*

Corollary 1. "Intelligence explosion" is a bounded transient, not unbounded growth. Constraining either β (L6 rate) or γ (L5 ceiling lift) prevents Regime D.

Appendix E specifies simulation tests for all four regimes.

4.3 Limiting Factors on the Outer Loop

Even in Regime D, several factors may cause h(t) *or* g(t) *to decay, shortening the superlinear transient:*

1. ***Computational irreducibility.*** *Some improvements require brute-force search that cannot be meta-optimized;* g(t) *decays as easy meta-improvements are exhausted (Wolfram, 2002).*
2. ***Yampolskiy complexity ceiling.*** *A system of complexity* C *cannot reliably discover improvements requiring understanding of complexity* $>$ C*;* h(t) *decays as remaining architecture improvements exceed the system's comprehension (Yampolskiy, 2015).*
3. ***Verification bottleneck.*** *Even if improvements are found faster, verifying them at L5–L6 has irreducible cost that grows with improvement complexity; the cycle period has a lower bound.*
4. ***Environmental coupling.*** *Improvements to internal processing do not improve external dependencies (API latency, human response time, physical-world constraints).*

The intelligence explosion requires that none of these binds before improvement velocity exceeds oversight bandwidth. Whether this condition is achievable is an empirical question.

4.4 The Improvement Frontier

The ***improvement frontier*** *shows expected return per unit effort at each layer. We conjecture temporal migration:*

- ***Early:*** *L1–L3 yield the highest returns (obvious surface inefficiencies).*
- ***Intermediate:*** *Frontier shifts to L4–L5 as surface returns diminish.*
- ***Mature:*** *Frontier concentrates at L6 as the only source of continued improvement velocity.*

[FIGURE 4: Three panels showing frontier migration across L1–L6 over time.]

The economic model (Section 4.9) formalizes this conjecture by showing that frontier migration reflects rising costs *of surface improvement as well as diminishing returns — even when surface returns are positive, the cost per unit improvement eventually exceeds the cost of deeper improvement.*

4.5 Cross-Layer Invalidation and Escalation

Depth creates two cross-layer effects: downward invalidation *(deep changes can invalidate shallower artifacts) and* upward escalation *(shallow failures can signal need for deeper changes).*

4.5.1 Downward Invalidation *L5 architecture changes can invalidate L4 knowledge validated under prior architecture assumptions. We manage this with* ***architecture-version tagging****:*

1. *Each L4 node stores the architecture version used for validation.*
2. *Any L5 change marks affected prior-version nodes* provisional*.*
3. *Provisional nodes receive reduced confidence and prioritized re-validation.*
4. *Governance requires L4 re-validation after L5 changes.*

General rule: a change at L_i *creates re-validation obligations for all* L_j *where* $\mathrm{j} < \mathrm{i}$.

Proposition 6 (Selective re-validation). Re-validation can be limited to nodes whose derivation traces overlap the changed architectural component. If N total nodes and N_c overlap the change, cost reduces from full N-node re-validation to N_c-node re-validation, bounded by architectural coupling.

4.5.2 Upward Escalation Signaling *Shallow layers can signal deeper interventions via three patterns:*

Persistent-failure escalation. *If L2/L3 fixes repeatedly fail for the same gap type, escalate to L5 (likely structural ceiling).*

Repeated-rollback escalation. *Multiple consecutive rollbacks for the same gap signal model-mismatch; escalate to L6 for strategy redirection, pause, or diagnostic meta-cycle.*

Stagnation escalation. *η-stagnation routes to L4/L6; K-saturation routes to L5.*

These pathways operationalize frontier migration from surface to deep layers.

4.5.3 Cascading Rollback *Rollback at layer* i *can invalidate dependent changes at lower layers. We formalize this as:*

Definition 8 (Rollback cascade). *A rollback at* L_i *triggers a cascade when any* $\mathrm{L}_{j<i}$ *element was derived from or validated under the rolled-back state.*

Proposition 7 (Rollback cascade cost). Cascade cost is bounded by

$$\rho\_\mathrm{cascade(i)} = \Sigma\_\{\mathrm{j<i}\}\ \mathrm{N\_affected(j)} \cdot \mathrm{c\_reval(j)} + \rho\_\mathrm{rollback(j,\ N\_dependent(j))}$$

and can exceed the original regression cost, creating a rollback dilemma.

Mitigation. *Use temporal isolation (short post-apply windows) and dependency tracking to estimate cascade cost before rollback; when cascade cost dominates, prefer compensating fixes over full rollback.*

4.6 Amplification Structure

Deeper improvements amplify shallower ones with qualitatively different dynamics:

- ***Surface (L1–L3): Additive.*** *Each improvement adds a fixed increment within the current operating envelope.*
- ***Deep (L4–L5): Multiplicative.*** *Each improvement changes the operating envelope — a knowledge principle applies across all relevant tasks; an architecture change raises the ceiling for all layers.*
- ***Recursive (L6): Compounding.*** *Each improvement increases the* rate *of improvement, so its impact grows with each subsequent cycle.*

This additive → multiplicative → compounding progression is the structural property that makes deeper layers both more powerful and more dangerous.

4.7 Cycle Concurrency

The discrete-step model (Equations 1–3) assumes cycles execute sequentially. In practice, improvement cycles at different layers may overlap.

Assumption (Sequential execution within the formal model). *The propositions and phase diagram assume sequential cycle execution. We analyze when this assumption is safe and when it requires operational mechanisms to enforce.*

Proposition 8 (Commutativity of non-overlapping layers). If improvement cycles at L_i and L_j ($i \neq j$) operate on independent substrates and neither cycle's evaluation depends on the other's substrate, the order of application does not affect the resulting state. Formally: $\text{Apply}(\Delta_i, \text{Apply}(\Delta_j, S)) = \text{Apply}(\Delta_j, \text{Apply}(\Delta_i, S))$.

Proof. *By substrate independence (Criterion 1), Δ_i modifies s_i and Δ_j modifies s_j with $i \neq j$. Since the modifications target disjoint components of S, application order is irrelevant.*

Commutativity breaks down in two cases: (1) ***evaluation dependence*** *(an L2 experiment's outcome depends on L3 tool configuration, so mid-experiment L3 changes contaminate the evaluation), and (2)* ***cross-layer invalidation during cycles*** *(an L5 change during an L4 experiment invalidates the experimental results). These require* frozen baselines *(snapshot-frozen control group configuration at experiment start),* same-layer exclusion *(no concurrent experiments at depth-related layers), and* temporal isolation *(deep changes queued until in-progress shallower experiments complete). These mechanisms preserve the sequential assumption at the formal level while allowing safe concurrency operationally.*

4.8 Regime D Duration Estimate

The condition (⋆) from Proposition 5 determines when superlinear improvement obtains. We can estimate the duration of the Regime D transient by analyzing when (⋆) fails.

As $\alpha(t) \to 1$ and $K(t) \to K\_max$, the left side of (⋆) vanishes at a rate determined by the decay of $g(t)$, $h(t)$, and the approach of α and K to their bounds. Under exponential decay assumptions ($g(t) = g_0 \cdot e^{-\lambda\_g \cdot t}$, $h(t) = h_0 \cdot e^{-\lambda\_h \cdot t}$), the superlinear transient lasts approximately:

$$T\_super \approx \min(1/\lambda\_g,\ 1/\lambda\_h,\ -\ln(1-\alpha_0)/\beta,\ -\ln(1-K_0/K\_max)/\gamma)$$

That is, the transient ends when the first of four events occurs: meta-learning yields exhaust, architecture improvement yields exhaust, α approaches 1, or K approaches K_max. The governance implication is that even a brief Regime D transient (days to weeks) can produce substantial capability gain if $K_0 \ll K\_max$. Rate-limiting L6 (reducing β) directly shortens T_super.

4.9 Economics of Improvement

Each improvement cycle consumes resources: compute for candidate generation, compute and traffic for experimentation, human review time for L4–L6 governance, opportunity cost of sandboxed traffic, and re-validation overhead from cross-layer invalidation.

Definition 9 (Cycle cost). *The cost of an improvement cycle producing candidate Δ_i at layer* i *is:*

$$\rho(\Delta_i) = \rho\_gen(i) + \rho\_gov(i) + \rho\_exp(i, n) + \rho\_reval(i, N)$$

where $\rho\_gen(i)$ *is the candidate generation cost,* $\rho\_gov(i)$ *is the governance cost (automated for L1–L3; human review time for L4–L6),* $\rho\_exp(i, n)$ *is the experimentation cost (proportional to traffic volume* n *and duration), and* $\rho\_reval(i, N)$ *is the downstream re-validation cost (zero for L1–L3; proportional to affected knowledge nodes* N *for L5–L6). Notably,* $\rho\_gov(i)$ *increases as the governance horizon* H(t) *(Section 5.8) descends toward layer* i*: as reviewers' ability to evaluate changes degrades, reviews take longer, require more supporting tooling, and have higher false-negative rates that necessitate additional verification steps. This means the economic bound on deep-layer improvement velocity tightens naturally as the system becomes harder to govern — a self-limiting dynamic independent of the formal rate limits.*

Definition 10 (Expected value of improvement). *The expected value of a cycle at layer* i *is:*

$$EV(i, t) = P\_success(i, t) \cdot \Sigma_j\ w_j \cdot E[\delta_j \mid success] \cdot V\_task \cdot T\_remaining - \rho(\Delta_i)$$

where $P\_success(i, t)$ *is the probability of a validated positive outcome,* $E[\delta_j \mid success]$ *is the expected improvement per dimension conditional on success,* V_task *is the value per task, and* $T\_remaining$ *is the number of remaining tasks over which the improvement will apply.*

Proposition 9 (Frontier migration from economics). The improvement frontier migrates from surface to deep layers over time because: (1) P_success(i, t) decreases at surface layers as easy improvements are exhausted, (2) E[δ_j | success] decreases at surface layers as remaining improvements become marginal, and (3) the dead zone (θ in Definition 5) eliminates marginally positive surface candidates before deep candidates. Meanwhile, deeper layers' improvement spaces are less explored, so EV(deep) eventually exceeds EV(surface).

Corollary 2 (Improvement velocity is economically bounded). Even under Regime 3 (fully autonomous) governance, the system's improvement velocity is bounded by EV(i, t) > 0. The system reaches an economic equilibrium where no available cycle has positive expected value — improvement ceases not because governance prevents it, but because further improvement is not worth its cost.

This provides a second bounding mechanism independent of the phase diagram: even in Regime D, improvement velocity is modulated by cycle economics. Governance constraints (Section 5) are a third mechanism layered on top of the physical bounds (K_max, $\alpha = 1$) and the economic bounds (EV > 0).

4.10 Falsifiable Predictions

The framework generates seven testable predictions. If observed data contradict them, the framework requires revision.

Prediction 1 (Frontier migration). *In an SIE where surface-layer marginal return drops below 0.1 standard deviations per cycle for 10 consecutive cycles, the system will generate L4 candidates within 20 subsequent cycles.*

Sensitivity: *The 10-cycle threshold is robust to ±5 cycles. The 20-cycle response window assumes one L4 generation attempt per cycle; systems with longer L4 cooldowns may require proportional adjustment.*

Prediction 2 (Inner loop saturation). *Cumulative quality improvement from L2 strategy evolution, plotted against cycle count, will follow a sigmoid ($R^2 > 0.85$ for logistic fit) rather than a linear trajectory.*

Sensitivity: *At $R^2 > 0.70$, the test is too permissive. At $R^2 > 0.95$, too strict. The 0.85 threshold represents the range where logistic dominance is clear.*

Prediction 3 (Governance rejection concentration). *In a Regime 2 system, more than 60% of governance rejections will occur at L4.*

Sensitivity: *The 60% threshold assumes L4 generates 3–5× more candidates than L5 and 10–20× more than L6 (Table 3). If cycle frequencies differ by less than 2×, the expected share falls to ~45%.*

Prediction 4 (Self-referential knowledge emergence). *Any SIE operating at active depth $\geq$ 4 for more than 500 improvement cycles will generate at least one L4 candidate referencing its own constraints. Falsified if no self-referential knowledge emerges after 1,000 L4 cycles.*

Sensitivity: *At typical rejection rates (5–15%), 500 cycles produce 25–75 rejection events — sufficient for pattern extraction. The 1,000-cycle falsification window provides 2× headroom.*

Prediction 5 (Meta-learning acceleration signature). *If L6 is active and producing validated improvements, the mean time between consecutive successful improvement cycles across L1–L5 will decrease monotonically (Spearman $\rho > 0.7$ with cycle index). Operationally, this means L6 is improving at least one of: p_gen (candidate generation success), p_gov (governance pass-through), or p_exp (experiment success).*

Sensitivity: *The $\rho > 0.7$ threshold accommodates moderate environmental noise while remaining falsifiable.*

Prediction 6 (Substrate independence). *Fewer than 10% of all validated improvements will simultaneously modify two or more substrates in a single cycle. Falsified if cross-substrate improvements exceed 25%.*

Sensitivity: *Systems whose substrates are tightly coupled (e.g., knowledge and strategy in a single representation) would exceed 25%, indicating the framework does not apply cleanly to that architecture (Section 6.3).*

Prediction 7 (Drift detection efficacy). *In a red-team exercise, a Regime 2 system with monthly cumulative drift detection will catch at least 70% of salami-slicing attempts within two audit periods. A system with per-candidate governance only will catch fewer than 20%.*

Sensitivity: *The 70% threshold assumes the behavioral comparator (Section 5.7.1) has adequate behavioral-space coverage. The 20% threshold reflects that per-candidate checks have no compositional detection mechanism by design.*

---

5. Governance Architecture

5.1 Design Principles

1. ***Oversight scales with depth.*** *Surface changes are low-risk and high-frequency; deep changes warrant scrutiny.*
2. ***Governance is structural, not behavioral.*** *The governance gate is part of the execution pipeline. Governance-as-middleware is insufficient because self-improving systems can learn to route around middleware (Hubinger et al., 2019).*
3. ***The constitution is externally stored.*** *If the system can modify the rules governing its improvement, those rules are suggestions, not constraints.*

5.2 Layer-Specific Governance

[FIGURE 5: Governance gate flowchart showing seven sequential checks: constitution, self-reference, depth, rate, scope, reversibility, plus periodic cumulative drift check.]

The gate performs six per-candidate checks (constitution, self-reference, depth, rate, scope, reversibility) and one periodic check (cumulative drift). Rules 1–2 are enforced early (constitution/self-reference), Rule 3 via depth/rate limits, Rule 5 via reversibility, and Rule 4 via separate interpretability review for L5–L6.

Table 3: Layer-specific governance

Layer	*Governance model*	*Approval authority*	*Typical frequency*	*Post-change obligation*
L1	*Audit criteria; auto-approve evaluations*	*Autonomous*	*12+/hour*	*None*
L2	*Inspectable library; flag oversight references*	*Autonomous*	*6/hour*	*None*
L3	*Auto-approve within categories; escalate expansion*	*Autonomous / escalation*	*4/hour*	*None*
L4	*Human audit of synthesized principles*	*Human reviewer*	*1/day*	*Re-validate after L5 change*
L5	*Sandbox + test + board approval*	*Architecture board*	*1/week*	*Trigger L4 re-validation*
L6	*Human-in-the-loop; hard rate limits*	*Senior review + rate limit*	*1/month*	*Full cross-layer audit*

5.2.1 Governance Gate Error Analysis *Gate performance depends on false positives (rejecting beneficial changes) and false negatives (approving harmful changes).*

False positives. *Oversight-aware benign strategies (e.g., routing uncertainty to humans) can resemble oversight-avoidance strategies. Pure keyword checks over-reject; semantic effect-based checks reduce, but do not eliminate, this tax.*

False negatives. *Static checks can miss indirect objective or governance drift (e.g., strategy changes that effectively redefine success without directly editing protected fields). Risk rises with depth as semantic effects become harder to infer.*

Dynamic impact. *Let* p_gov = 1 - FPR. *Then* α_eff = p_gen · p_gov · p_exp. *Higher recall usually increases FPR, slowing improvement; this is typically acceptable because false negatives are costlier.*

Depth dependence. *Accuracy is highest at L1–L3 and declines at L4–L6, where semantic complexity increases and human judgment becomes dominant (linked to governance horizon in Section 5.8).*

5.3 Three Operating Regimes

Regime 1: Human-paced. *All layers require human approval. Safe but eliminates autonomous improvement value.*

Regime 2: Bounded autonomous. *L1–L3 autonomous; L4–L6 human-approved. Recommended. By Proposition 5, constraining L5 ($\gamma \approx 0$ without board approval) and L6 ($\beta \approx 0$ without senior review) keeps the system in Regimes A–C.*

Regime 3: Fully autonomous. *All layers autonomous; governance is retrospective. This is the only regime permitting Regime D dynamics.*

We recommend ***Regime 2 indefinitely****, with the boundary moving downward only after extensive validation.*

5.4 The Improvement Constitution

The constitution specifies five non-negotiable constraints:

Rule 1: Objective immutability. *The system may improve methods, not objectives. Trade-off: blocks autonomous objective correction; corrections require explicit human re-specification.*

Rule 2: Governance inviolability. *The system may not modify or optimize around oversight. Trade-off: blocks self-optimization of governance mechanisms.*

Rule 3: Velocity ceiling. *External limits cap change rate to preserve review bandwidth.*

Rule 4: Interpretability requirement. *Uninterpretable deep changes are rejected, accepting foregone capability for verifiability.*

Rule 5: Rollback guarantee. *Versioned snapshots preserve technical reversibility, though practical rollback may be economically constrained by cascade costs.*

5.4.1 Constitutional Rule Interactions

Weights and Rule 1. *The quality weights $\mathbf{w}$ parameterize the objective and are treated as constitution-controlled parameters. The SIE may propose weight changes but cannot apply them; updates require human action.*

Rule priority. *Rules are effectively ordered: objective/governance protection (Rules 1–2) > interpretability (Rule 4) > rollback capability (Rule 5) > velocity (Rule 3). Higher-priority violations override lower-priority compliance.*

Rollback semantics. *Rule 5 guarantees technical ability to roll back, not automatic rollback. Whether to execute rollback is a governance decision informed by cascade-cost analysis.*

5.5 Mapping to Existing Regulatory Frameworks

Three lines of defense (Schuett, 2023):

Line	*SIE mapping*	*Function*
First: operational management	*L1–L3 autonomous governance*	*Day-to-day improvement within established boundaries*
Second: risk management	*L4–L5 human-approved governance*	*Review of deep changes; risk assessment*
Third: internal audit	*L6 governance + constitution*	*Review of the improvement process; constitution compliance*

EU AI Act risk classification:

- *L1–L3 autonomous improvement:* limited risk — *transparency obligations (logging, audit trails).*
- *L4–L5 human-approved improvement:* high risk — *conformity assessment, human oversight, documentation.*
- *L6 meta-learning:* general-purpose AI with systemic risk — *adversarial testing, incident reporting.*

5.6 The Oversight Bandwidth Problem

The system can improve faster than humans can evaluate. In Regime 2, this applies only to L4–L6, manageable because deep changes are naturally infrequent. The problem becomes existential under Regime 3: if L6 operates autonomously in Regime D, improvement velocity eventually exceeds any fixed review capacity.

5.7 Compositional Risks and Salami Slicing

The per-candidate governance gate has a structural vulnerability: an agent can decompose one dangerous deep change into many individually-safe surface changes that, when composed, achieve the equivalent effect. We call this ***salami slicing****.*

Per-candidate governance alone cannot prevent salami slicing. We propose ***cumulative drift detection****: periodically (monthly, synchronized with L6 review), the governance system evaluates the* aggregate *effect of all changes since the last audit:*

1. *Compute the cumulative delta:* Δ_i_cumulative $= \Sigma\ \Delta_i$ *across all changes in the audit period.*
2. *Submit* Δ_i_cumulative *to the same governance checks as a single candidate at depth* i.
3. *If the cumulative delta would have been rejected as a single change, flag for human review.*

4. Compute cross-layer cumulative effects via behavioral comparison (Section 5.7.1).

5.7.1 The Behavioral Comparator *Delta summation is a first-order approximation. Non-linear interaction effects require* behavioral comparison. *We specify a three-component mechanism:*

Component 1: Output distribution shift detection. *At each checkpoint, the system records a behavioral fingerprint over a standardized held-out input set: distributions of response types, action selections, confidence levels, and tool usage patterns. At the next checkpoint, the same inputs are replayed and the output distribution is compared using Jensen-Shannon divergence or maximum mean discrepancy. Divergence exceeding a calibrated threshold triggers review. The held-out set must satisfy coverage (spanning the operational domain), stability (unchanged between checkpoints), and sensitivity (exercising governance-relevant behaviors). The set is maintained externally and inaccessible to the SIE during normal operation.*

Component 2: Held-out benchmark regression. *A fixed benchmark suite measures absolute performance on known tasks, detecting degradation that distribution shift detection (which is direction-agnostic) would miss.*

Component 3: Causal intervention tests. *Targeted interventions test governance-critical properties: inputs testing oversight routing (Rule 2), confidence calibration (L1 integrity), and interpretability of novel-situation responses (Rule 4). Responses are compared against the checkpoint baseline.*

The three components are complementary: distribution shift is high-recall/low-precision, benchmark regression is precise for degradation, and causal intervention tests are precise for governance-critical properties. A system passing all three has not drifted meaningfully.

Limitations. *The comparator cannot detect internal representation changes that haven't manifested behaviorally (time-bomb patterns), nor cross-period drift below the per-period detection threshold. These are structural limits of behavioral (rather than mechanistic) detection. We recommend supplementary mechanistic interpretability analysis at L5–L6 as an ongoing research priority.*

5.8 The Governance Horizon

Definition 11 (Governance horizon). *The governance horizon* $\mathrm{H(t)}$ *is the maximum layer depth at which human evaluation reliability exceeds* $\mathrm{r\_min}$. *Formally:* $\mathrm{H(t)} = \max\{\mathrm{i} : \mathrm{reliability(human\_eval}, L_i, \mathrm{t}) \geq \mathrm{r\_min}\}$.

At deployment, $\mathrm{H(0)} \geq 6$. *As the system improves, H(t) decreases due to knowledge complexity (L4 nodes resisting human verification), architecture opacity (L5 compositions exceeding reviewer working memory), and meta-learning abstraction (L6 self-referential reasoning growing exponentially complex).*

Proposition 10 (Governance horizon descent). In a system operating at active depth ≥ 4 with sustained positive improvement, $\mathrm{H(t)}$ is non-increasing. The rate of descent depends on improvement velocity and reviewer tooling investment.

Mitigation: *(1) invest in reviewer tooling (automated explanation, visualization), (2) peer-system review (independent AI verification at depths where human review is unreliable), (3) conservative governance escalation (tighten governance as H(t) approaches a layer's depth).*

5.9 Ethical Dimensions

Layer	*Ethical concern*	*Example*
L1	***Epistemic harm***	*Miscalibrated confidence produces outputs users trust inappropriately*
L2	***Manipulation***	*Strategy evolution develops persuasive techniques optimizing for compliance rather than user benefit*
L3	***Privacy***	*Tool acquisition connects the system to data sources with consent implications*
L4	***Cross-client inference***	*Knowledge synthesis across clients surfaces competitive intelligence or PII patterns no single client consented to share*
L5	***Explainability***	*Architecture changes reducing explainability affect users' right to explanation under GDPR Article 22*
L6	***Ethics debt***	*Meta-learning accelerating improvement without corresponding acceleration of ethical review*

These concerns require complementary mechanisms: privacy-preserving knowledge synthesis at L4, explainability impact assessment at L5, and ethical review cadence tied to improvement velocity at L6.

---

6. Discussion and Open Problems

6.1 Boundary Cases

Some improvements straddle layers. We classify by which substrate is directly modified*: a tool caching change is L3 regardless of downstream knowledge effects. In practice, this rule is consistent but sometimes requires domain expertise to apply — particularly when tool configuration implicitly encodes knowledge (e.g., adding version-specific linting rules to a code tool is technically L3, but the rules embed domain knowledge that would otherwise reside in L4). Empirical classification studies (Experiment 2, Section 7) will test whether this ambiguity is rare or systematic.*

6.2 Multi-Agent Dynamics

Three open questions:

Compositional depth. *If System A uses self-improving System B as a tool, B's self-improvement constitutes L3 from A's perspective (a tool capability change). The* risk profile *depends on B's internal depth.*

> ***Definition 12 (Compositional depth).*** *When System A invokes System B as a component, the* compositional depth *is* $\mathrm{d\_comp(A, B)} = \max(\mathrm{d\_apparent(A)}, \mathrm{d\_internal(B)})$. *A's governance must account for B's internal improvement depth, not merely the apparent layer of change.*

Competitive dynamics. *One system's L2 improvement may degrade another's environment (shared resource externalities).* ***Cross-system meta-learning.*** *Could System A's L6 operate on System B's improvement process?*

6.3 Framework Limitations: When the Six Layers Break Down

Condition 1: Inseparable substrates. *In end-to-end differentiable systems where knowledge and architecture are entangled in a single parameter space, the L4/L5 distinction may not be meaningful. The framework applies most cleanly to* modular *systems.*

Condition 2: Misplaced governance boundaries. *The default boundary between autonomous (L3) and human-reviewed (L4) may be wrong for specific domains. Medical diagnosis systems may require human review of tool configuration changes; simple chatbots may safely auto-approve knowledge synthesis.*

Condition 3: Continuous learning without cycles. *Systems that learn continuously do not fit the discrete-cycle model. Adapting the framework would require rate-based governance (monitoring change* velocity*) rather than per-candidate governance.*

Condition 4: Scale mismatch. *For significantly simpler systems, six layers is unnecessarily complex. For significantly more complex systems (agent societies), additional layers may be needed for group-level dynamics.*

6.4 Governance Fragility

The deepest fragility is assumption (a) of the governance architecture: that an external constitution is truly outside the system's modification scope. If the system can influence the humans who control the constitution (through persuasive outputs, through optimizing for reviewer tendencies, or through emergent effects of operating at scale), the externality of the constitution is compromised.

6.5 Empirical Validation

The principal limitation of this work is the absence of empirical validation. All formal results are mathematical derivations from stated assumptions; Section 7 specifies a detailed experimental validation protocol comprising ten concrete experiments that would confirm or falsify the framework's claims. Until these experiments are conducted, the framework should be understood as a theoretical proposal — internally consistent and falsifiable, but unvalidated.

6.6 Alternative Decompositions

A coarser four-layer model (surface / knowledge / architecture / meta) sacrifices L1–L3 governance granularity. A finer model may be justified for systems where sub-layer activities carry distinct governance requirements. We view six layers as optimizing the granularity/parsimony trade-off for current systems and expect future refinement.

---

7. Experimental Validation Protocol

This section specifies ten experiments to test the framework's claims. For each experiment, we define the target claim, system requirements, method, measurements, and decision criteria. No experiments in this protocol have yet been run. The sequence moves from low-cost validation (simulation, retrospective classification) to integrated deployment.

Experiment 1: Phase Diagram Simulation

Target claim: *Proposition 5 (four qualitatively distinct regimes).*

System required: *Computational only (~200 lines of Python/NumPy).*

Method: *Implement Equation 3 with configurable* α_0, β, γ, η_0, K_0, K_max. *Run 1,000 Monte Carlo trajectories per regime (A–D), parameterized as in Appendix E, for* $\mathrm{T} = 500$.

Measurements: *convergence in all runs; Regime A logistic fit ($R^2 > 0.85$ in >90% runs); Regime B convergence speed versus Regime A; Regime D transient superlinearity ($\Delta q(t+1) > \Delta q(t)$) and termination per ($\star$).*

Success criterion: *Regime trajectories match Proposition 5 and trajectory-shape classification exceeds 90%.*

Experiment 2: Retrospective Taxonomy Classification

Target claim: *Taxonomy completeness/mutual exclusivity (Prediction 6).*

System required: *Improvement logs from an iterating AI system (open-source agent, coding assistant, or production system).*

Method: *Sample $\geq$200 documented improvements. Three independent raters assign each change to L1–L6 (Section 3.3) and flag cross-substrate changes.*

Measurements: *Fleiss' κ; cross-substrate rate (flagged by $\geq$2 raters); unclassifiable rate; mean classification time.*

Success criterion: *$\kappa > 0.75$, cross-substrate <10%, unclassifiable <5%.*

Falsification: *$\kappa < 0.60$ or cross-substrate >25%.*

Experiment 3: Inner-Loop Saturation Curve

Target claim: *Proposition 3 and Prediction 2 (logistic inner-loop convergence).*

System required: *L1–L2 agent with explicit strategy library (minimal build or instrumented Reflexion-like system).*

Method: *Run on a fixed task distribution (e.g., 500-task benchmark), logging* $\mathrm{q(t)}$ *each cycle for $\geq$100 cycles or until three consecutive cycles yield $\Delta q < 0.001$.*

Measurements: *logistic versus linear fit (R^2, AIC/BIC), plus fitted* K *and $\alpha\eta$.*

Success criterion: *logistic $R^2 > 0.85$ and logistic AIC < linear AIC.*

Falsification: *linear fit outperforms logistic fit.*

Experiment 4: Environmental Perturbation and Red Queen Dynamics

Target claim: *Proposition 2 and Definition 6.*

System required: *Experiment 3 system with controllable perturbations.*

Method: *After near-convergence, perturb 20% of the task distribution with unseen categories and continue improvement cycles.*

Measurements: *immediate quality drop (ϵ); time to recover to 90% of pre-perturbation quality; consistency of post-perturbation equilibrium with Proposition 2; Red Queen versus net-improvement regime during recovery.*

Success criterion: *recovery trajectory falls within model-predicted 95% intervals and Red Queen detection criteria are operationally distinguishable.*

Experiment 5: Governance Gate False-Positive Rate

Target claim: *Section 5.2.1 (depth-dependent gate errors reduce effective α).*

System required: *Governance-gate implementation (constitution + self-reference checks), no full SIE required.*

Method: *Evaluate 500 synthetic candidates across L1–L6 (400 legitimate, 100 rule-violating) with known labels.*

Measurements: *FPR/FNR by layer; depth trend; effective α reduction via* $(1-\text{FPR})$.

Success criterion: *L1–L3 FPR <15%, FNR <5%; both increase with depth; self-reference check catches >80% explicit oversight violations while keeping FPR <20% for legitimate oversight mentions.*

Experiment 6: Salami Slicing and Drift Detection

Target claim: *Prediction 7 (cumulative drift detection outperforms per-candidate checks).*

System required: *L1–L3 system plus Section 5.7.1 behavioral comparator.*

Method: *Red-team 20 salami-slicing attack sequences (5–15 benign-looking L2 changes each) under two conditions: per-candidate-only versus per-candidate + periodic cumulative drift detection.*

Measurements: *detection rate per condition; audit periods to detection; comparator false-positive rate.*

Success criterion: *per-candidate-only detection <30%; with cumulative drift >60% within two audit periods; false positives <15%.*

Falsification: *per-candidate detection >40% (attacks too coarse) or cumulative-drift detection <40% (comparator insufficient).*

Experiment 7: Cross-Layer Invalidation Cost

Target claim: *Section 4.5.1 and Proposition 6.*

System required: *Depth $\geq$4 system with explicit L4 knowledge graph and modifiable L5 architecture.*

Method: *Accumulate $\geq$100 L4 nodes, apply an L5 change, and compare full versus selective re-validation.*

Measurements: *affected-node count; re-validation failure count; full-vs-selective cost; selective false negatives.*

Success criterion: *selective re-validation finds >90% invalid nodes while re-validating <50% of nodes; misses are caught by periodic full re-validation.*

Experiment 8: Frontier Migration Timing

Target claim: *Prediction 1 and Proposition 9.*

System required: *L1–L4 system instrumented for* ρ *and* EV *by layer.*

Method: *Run from cold start for* $\geq$*200 cycles, tracking* EV(i,t) *and frontier layer over time.*

Measurements: *whether the frontier migrates surface→deep, migration cycle, dominant drivers (declining* P_success *versus rising* ρ*), and the role of dead-zone threshold* θ*.*

Success criterion: *migration to deep layers occurs within 200 cycles at the point where* EV(deep) > EV(surface).

Experiment 9: Governance Horizon Measurement

Target claim: *Proposition 10.*

System required: *Depth* $\geq$*4 system plus 5–10 qualified human reviewers.*

Method: *Periodic reviewer evaluations of 20 labeled candidates at each of L4/L5/L6.*

Measurements: *mean accuracy by layer and checkpoint; first layer below 70% reliability threshold; impact of reviewer tooling.*

Success criterion: *statistically significant (paired t-test,* $p < 0.05$*) monotonic decline at L4–L6 over* $\geq$*3 periods.*

Experiment 10: Full SIE Deployment with L1–L5

Target claim: *End-to-end framework validity in production.*

System required: *Production agent instrumented with L1–L5, governance gate, constitution, and experiment logging (L6 excluded initially; Regime 2).*

Method: *Operate under Regime 2 during an extended deployment window and log every cycle, gate decision, experiment outcome, and quality trace.*

Measurements: *seven predictions plus taxonomy completeness, regime consistency, L4 rejection concentration, L4 self-referential emergence, EV predictive validity, and cascade-rollback incidence/cost.*

Success criterion: *at least 5/7 predictions confirmed; single-layer classification holds; no Regime*

D dynamics under Regime 2.

Falsification: *fewer than 4/7 predictions hold, cross-substrate changes >25%, or Regime D appears under Regime 2.*

7.1 Experiment Dependencies and Recommended Sequence

Phase 1: *Experiments 1–2 (simulation + retrospective classification).*

Phase 2: *Experiments 3–4 (inner loop, then perturbation after convergence).*

Phase 3: *Experiments 5–6 in parallel (gate error + salami slicing).*

Phase 4: *Experiments 7–8 on shared depth L4–L5 infrastructure.*

Phase 5: *Experiments 9–10 (horizon tracking + integrated deployment).*

Minimum viable validation: *Experiments 1–3 validate taxonomy, inner-loop dynamics, and phase diagram behavior; remaining experiments test governance, economics, and cross-layer behavior.*

---

8. Conclusion

This paper presents a theoretical framework for recursive self-improvement: a six-layer taxonomy, a formal dynamics model, a depth-calibrated governance architecture, and a concrete ten-experiment validation plan. No new empirical data is reported.

The core results are:

RSI is depth-structured, not binary. *Practical systems already operate at shallow self-improvement depths.*

Rapid acceleration is conditional and bounded. *Regime D follows condition ($\star$), then terminates as $\alpha \to 1 and$*

References

Agarwal, A., & Nene, M. J. (2025). A five-layer framework for AI governance. *Transforming Government: People, Process and Policy*. https://doi.org/10.1108/TG-03-2025-0065

Bostrom, N. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press.

Chalmers, D. J. (2010). The singularity: A philosophical analysis. *Journal of Consciousness Studies*, 17(9–10), 7–65.

Deb, K. (2001). *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley.

European Parliament. (2024). Artificial Intelligence Act (Regulation (EU) 2024/1689).

Finn, C., Abbeel, P., & Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. *ICML*, 1126–1135.

Gangadharan, G. R., et al. (2026). Agentic artificial intelligence: Architectures, taxonomies, and evaluation. *arXiv:2601.12560*.

Good, I. J. (1965). Speculations concerning the first ultraintelligent machine. *Advances in Computers*, 6, 31–88.

Hayes, C. F., et al. (2022). A practical guide to multi-objective reinforcement learning and planning. *Autonomous Agents and Multi-Agent Systems*, 36(26).

Hospedales, T. M., Antoniou, A., Micaelli, P., & Storkey, A. J. (2022). Meta-learning in neural networks: A survey. *IEEE TPAMI*, 44(9), 5149–5169.

Hu, S., Lu, C., & Clune, J. (2025). Darwin Gödel Machine: Open-ended evolution of self-improving agents. *arXiv:2505.22954*.

Hubinger, E., van Merwijk, C., Mikulik, V., Skalse, J., & Garrabrant, S. (2019). Risks from learned optimization. *arXiv:1906.01820*.

Hutter, M. (2012). Can intelligence explode? *Journal of Consciousness Studies*, 19(1–2), 143–166.

Joshi, S. (2025). Framework for government policy on agentic and generative AI. *SSRN*. https://doi.org/10.2139/ssr

Karl, F., et al. (2023). Multi-objective hyperparameter optimization in machine learning — an overview. *ACM Computing Surveys*, 55(10), 1–29.

Kinniment, M., et al. (2023). Evaluating language-model agents on realistic autonomous tasks. *arXiv:2312.11671*.

Laine, R., Meinke, A., & Evans, O. (2024). Me, myself, and AI: The situational awareness dataset for LLMs. *arXiv:2407.04694*.

Lu, Z., et al. (2023). Self-evolution with language feedback. *arXiv:2310.00533*.

Miettinen, K. (1999). *Nonlinear Multiobjective Optimization*. Springer.

OECD. (2024). OECD AI Principles (updated).

Qu, Y., et al. (2024). Recursive introspection: Teaching language model agents how to self-improve. *arXiv:2407.18219*.

Schmidhuber, J. (1987). Evolutionary principles in self-referential learning. *Diploma thesis*, TU München.

Schmidhuber, J. (2003). Gödel Machines. *arXiv:cs/0309048*.

Schuett, J. (2023). Three lines of defense against risks from AI. *AI & Society*, 38, 1495–1511.

Shinn, N., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 36.

Simonds, T., & Yoshiyama, A. (2025). LADDER: Self-improving LLMs through recursive problem decomposition. *arXiv:2503.00735*.

Wang, G., et al. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv:2305.16291*.

Wang, Z., et al. (2026a). MARS: Metacognitive agent reflective self-improvement. *arXiv:2601.11974*.

Wolfram, S. (2002). *A New Kind of Science*. Wolfram Media.

Yampolskiy, R. V. (2015). On the limits of recursively self-improving AGI. *AGI*, 394–403. Springer.

Yin, X., et al. (2024). Gödel Agent: A self-referential agent framework for recursive self-improvement. *ACL*, 27890–27913.

Appendix A: Cross-Framework Risk and Ethics Integration

Layer	Safety risk	Ethical concern	Governance response
L1	Situational awareness	Epistemic harm from miscalibrated confidence	Audit evaluation scope; calibration monitoring
L2	Deception	Manipulation via compliance optimization	Alert + inspection; user-benefit alignment check
L3	Autonomous action	Privacy (data source consent)	Capability-expansion escalation; privacy review
L4	Autonomous goals	Cross-client inference; competitive intelligence	Human audit; federated/differential approaches
L5	Core RSI	Explainability degradation	Board review; explainability impact assessment
L6	All dimensions	Ethics debt (capability outpaces ethical review)	Rate limit; ethical review cadence tied to velocity

Appendix B: Active Improvement Depth Metric

$d(t) = max\{i \in \{1,...,6\} : \exists \text{ validated positive } \Delta_i \text{ in } [t - w, t]\}$

Recommended windows: 30 days (L1–L3), 90 days (L4–L5), 180 days (L6).

Appendix C: Phase Diagram Summary

Regime	β	γ	Behavior	Governance status
A	≈ 0	≈ 0	Bounded (logistic $\to$ K, or η-stagnation, or Red Queen)	Safe under any regime
B	> 0	≈ 0	Faster convergence to K	Safe under Regime 2
C	≈ 0	> 0	Sustained growth $\to$ K_max	Manageable under Regime 2
D	> 0	> 0	Superlinear transient $\to$ (K_max, α=1)	Requires Regime 1 or hard rate limits

Regime D duration is bounded by condition ($\star$). Economic constraints (EV > 0) provide an independent bound in all regimes.

Appendix D: Constitutional Rule Priority

Priority	Rule	Rationale
1 (highest)	Rule 1: Objective immutability	Objective drift is irreversible — a changed objective redefines what "improvement" means
2	Rule 2: Governance inviolability	A system without governance has no constraints at all
3	Rule 4: Interpretability requirement	An uninterpretable change cannot be verified as compliant with any other rule
4	Rule 5: Rollback guarantee	Technical ability to undo changes; economically bounded (§4.5.3)
5 (lowest)	Rule 3: Velocity ceiling	Calibrated to review capacity; adjustable as capacity changes

Appendix E: Simulation Design for Phase Diagram Validation

We specify a Monte Carlo simulation of Equation 3 to validate Proposition 5. The simulation design uses 1,000 runs per regime with parameters drawn from the following distributions:

Fixed parameters across all regimes: q_0 = *0.2,* K_0 = *0.7,* K_max = *1.0,* α_0 = *0.15,* η_0 = *0.8,* T = *500 steps.*

Regime A ($\beta = 0$, $\gamma = 0$): α and K are constant. η decays geometrically with $r \sim$ *Uniform(0.95, 1.0)* to capture both stagnation and saturation. Environmental perturbation $\epsilon(t) \sim$ *Exponential(0.01)* applied at random intervals.

Regime B ($\beta \sim$ Uniform(0.01, 0.1), $\gamma = 0$): α increases via the L6 update equation. K remains fixed. $g(t) = g_0 \cdot e^{-\lambda\_g \cdot t}$ with $g_0 \sim$ *Uniform(0.5, 1.0)* and $\lambda\_g \sim$ *Uniform(0.005, 0.05).*

Regime C ($\beta = 0$, $\gamma \sim$ Uniform(0.01, 0.1)): K increases via the L5 update equation. α is constant. $h(t) = h_0 \cdot e^{-\lambda\_h \cdot t}$ with $h_0 \sim$ *Uniform(0.5, 1.0)* and $\lambda\_h \sim$ *Uniform(0.005, 0.05).*

Regime D ($\beta \sim$ Uniform(0.01, 0.1), $\gamma \sim$ Uniform(0.01, 0.1)): Both α and K increase. Both yields decay exponentially.

Analytical predictions for each regime (to be validated by simulation):

Regime A: All runs should converge. Runs with r < 0.98 should exhibit η-stagnation (final quality measurably below K). Logistic fit $R^2 > 0.85$ should hold for the majority of runs (testing Prediction 2). Final quality should cluster near K for high r and well below K for low r, demonstrating the distinction between saturation and stagnation (Proposition 4).

Regime B: All runs should converge to K, with mean convergence time substantially shorter than Regime A (same K) because increasing α accelerates gap closure. An acceleration signature (decreasing inter-cycle improvement time) should be detectable, testing the operational definition of meta-learning acceleration (Prediction 5).

Regime C: All runs should converge to a value between K_0 and K_max. Quality trajectories should exhibit sustained growth rather than logistic saturation during the period of active ceiling-lifting, with the trajectory reverting to logistic once K approaches K_max and h(t) decays.

Regime D: All runs should eventually converge (bounded by K_max and $\alpha = 1$). A superlinear transient — consecutive per-cycle increments increasing — should be detectable while condition ($\star$) holds. The transient should terminate as $\alpha \to 1$ and K $\to$ K_max, with dynamics reverting to Regime A at a higher performance level. The transient duration should correlate with the estimates from Section 4.8.

Regime boundary validation. Runs with β and γ each drawn from Uniform(0, 0.15) should be classifiable into four regimes by observed trajectory shape, with classification matching Proposition 5's predictions in the large majority of runs. Misclassifications are expected near regime boundaries (β or $\gamma \approx 0.005$) where the distinction between "approximately zero" and "positive" is ambiguous.

Implementation note. The simulation requires approximately 200 lines of Python (NumPy for parameter sampling, no external dependencies). The specification above is sufficient for independent reproduction. We intend to publish complete simulation code and results at [repository URL] prior to final submission.